

Methode de Benders adverse étendue pour un problème de planification robuste

Sommaire

5.1	Introduction et description du problème	89
5.2	Revue de littérature	91
5.3	Méthode de Benders adverse (BA)	92
5.3.1	Le problème minmax	93
5.3.2	Le problème Adv	94
5.3.3	Algorithme de Benders adverse pour le lot-sizing incertain	96
5.4	Méthode de Benders adverse étendue (BAE)	96
5.4.1	Graphe partiel de budget	97
5.4.2	Problème maître	97
5.4.3	Sous-problème (Adv)	98
5.4.4	Fusion de graphes partiels de budget	99
5.4.5	Algorithme de Benders adverse étendue pour le lot-sizing incertain	100
5.5	Expérimentations numériques	101
5.5.1	Description des instances	101
5.5.2	Sélection des noeuds	101
5.5.3	Résultats expérimentaux	102
5.6	Conclusion et perspectives	103

Les travaux présentés dans ce chapitre ont été présentés à la conférence ROADEF 2022 et ont été finalistes du prix du meilleur article étudiant [Portoleau 2022b].

5.1 Introduction et description du problème

Dans les deux chapitres précédents, nous avons proposé un modèle permettant d'approcher de manière robuste et optimiste des problèmes d'ordonnancement incertains, dans la mesure où un décideur a accès à des informations à propos du scénario alors que celui-ci est en train de se réaliser.

Dans ce chapitre, nous proposons une variante d'une méthode d'optimisation robuste appelée méthode de Benders adverse, et l'appliquons non pas à un problème d'ordonnancement mais à un problème de gestion de production. La méthode de Benders adverse (ou parfois méthode des plans coupants) a été introduite dans [Mutapcic 2009]. Cette approche se base notamment sur la résolution itérée d'un problème dit adverse, c'est-à-dire problème où l'on cherche à calculer un pire scénario pour une solution réalisable donnée. Une approche similaire appelée "algorithme de relaxation des scénarios" avait été proposée pour des problèmes d'optimisation robuste en version min-max regret dans [Assavapokee 2008]. La méthode de Benders adverse est indiquée lorsque ce problème adverse est facile à résoudre et dépend entre autres de la nature de l'ensemble des scénarios. Dans [Pätzold 2020], les auteurs présentent une variante de cette méthode où des pires scénarios sont calculés de manière approximative, et prouvent la convergence de leur méthode. L'intuition de la méthode que nous présentons dans ce chapitre, est que l'on cherche à calculer non pas un seul scénario pire cas mais un ensemble de pires scénarios, avec l'idée que l'on peut gagner du temps de calcul en réduisant le nombre d'itérations de l'algorithme. Nous présentons cette méthode pour un problème de dimensionnement de lots (*lot-sizing*) avec contraintes de capacité (*Capacited Lot Sizing Problem* – CLSP), sous budget d'incertitude portant sur la demande cumulée. Typiquement, pour modéliser ce problème, on se donne un horizon découpé en T périodes de temps, et on associe à chaque période $t \leq T$ une demande d_t , et une production x_t . L'ensemble de ces variables de production constitue un plan de production. Le coût de production d'une période t est un coût de lancement c^P . A ce coût, des coûts linéaires liés à chaque pas de temps t et aux trois quantités suivantes sont ajoutées. Tout d'abord, si l'on produit plus que la demande, le surplus stocké sera noté I_t . Ensuite, la quantité effectivement vendue, qui satisfait au moins une partie de la demande est notée s_t . Et finalement, la quantité B_t représente la part de la demande non satisfaite. Ces trois quantités sont pondérées, respectivement par le coût de stockage c^I , le prix de vente b^P et le coût de réapprovisionnement c^B . On supposera que la quantité produite à chaque pas de temps est bornée, et donc que $X = (x_t)_{t \leq T}$ prend ses valeurs dans un polytope $\mathbb{X}$. On peut noter que ces différents coûts sont indépendants de la période de temps et que ce problème de lot sizing déterministe est NP-Difficile [Bitran 1982]. L'incertitude du problème porte sur la demande cumulée. Pour modéliser cela, on pose $X_t = \sum_{i=1}^t x_i$ et $D_t = \sum_{i=1}^t d_i$, qui représentent respectivement, la production cumulée jusqu'à la période t et la demande cumulée jusqu'à t . Concernant les incertitudes des demandes cumulées, on suppose que chaque demande D_t prend sa valeur dans un intervalle symétrique de la forme $[\hat{D}_t - \Delta_t, \hat{D}_t + \Delta_t]$, avec $\hat{D}_t \geq \Delta_t \geq 0$ où $\hat{D}_t$ est sa valeur nominale et Δ_t sa déviation maximale. On appelle scénario une réalisation de ces incertitudes et on note $\mathcal{U}$ un ensemble de scénarios. Dans la suite, on s'intéressera plus particulièrement à l'ensemble de scénarios continus suivant :

$\mathcal{U}$ est l'ensemble des vecteurs de demandes cumulées $\mathcal{D} = (D_t)_{t \leq T}$ vérifiant les

contraintes suivantes :

$$\begin{aligned}
 D_t &\leq D_{t+1} & t &\leq T-1 \\
 D_t &\geq \hat{D}_t - \Delta_t & t &\leq T \\
 D_t &\leq \hat{D}_t + \Delta_t & t &\leq T \\
 \|\mathcal{D} - \hat{\mathcal{D}}\|_1 &\leq \Gamma
 \end{aligned}$$

où Γ est le budget d'incertitude. Pour résoudre ce problème de lot-sizing nous avons choisi d'utiliser le critère de robustesse absolue, ou *minmax*. Naturellement puisque le problème déterministe est déjà difficile, il le reste dans sa version incertaine.

5.2 Revue de littérature

Le problème de *lot-sizing* sous incertitude a vastement été étudié au cours des dernières décennies, et l'on se limitera dans cette section à des références où l'incertitude porte sur la demande. Tout d'abord, on trouve dans la littérature un certain nombre d'articles où l'incertitude du problème est décrite par des arbres de scénarios. C'est le cas de [Brandimarte 2006], dans lequel les auteurs considèrent une version du problème avec capacité multi-éléments. L'incertitude de la demande est explicitement modélisée par un arbre de scénarios, ce qui donne lieu à un modèle de programmation stochastique en nombres entiers mixtes à plusieurs étapes avec recours. Ils proposent une formulation du modèle basée sur la localisation des usines et une approche heuristique basée sur une recherche locale. Ils présentent des expérimentations numériques pour évaluer non seulement la viabilité de l'heuristique, mais aussi l'avantage (le cas échéant) du modèle de programmation stochastique par rapport au modèle déterministe considérablement plus simple basé sur l'espérance de la demande. Ils essayent différentes approches pour générer l'arbre de scénarios. Leurs résultats suggèrent qu'il existe une interaction entre différents leviers de gestion pour couvrir l'incertitude de la demande, c'est-à-dire les tampons de capacité réactive et les stocks de sécurité. Lorsque la capacité réactive est suffisante, la capacité du modèle stochastique à constituer des stocks de sécurité est de peu d'utilité. Lorsque la capacité est fortement limitée et que l'impact des temps de préparation est important, des avantages remarquables sont obtenus en modélisant explicitement l'incertitude. Avec des objectifs très similaires, dans [Hu 2016] les auteurs traitent d'un problème de *lot-sizing* multi-produits avec des configurations dépendantes de la séquence. Ils proposent un modèle de programmation stochastique en deux étapes pour minimiser les coûts totaux de production, de stock et de rupture de stock. La première étape décide de la production de base, y compris la quantité de production de chaque produit et la séquence de production, tandis que la deuxième étape se concentre sur les mises à jour possibles de la production de base, telles que la production d'heures supplémentaires. L'objectif est de trouver la meilleure séquence de quantités de production sous une demande aléatoire avec des commandes en attente autorisées. Là aussi, l'incertitude est explicitement représentée par un arbre de scénarios puis par la sélection des scénarios les plus représentatifs afin d'obtenir un sous-ensemble plus petit tout en préservant les propriétés essentielles. Le temps de préparation et le coût de prépara-

tion dépendent tous deux du produit. Une étude de cas pour une entreprise de fabrication d'équipements de freinage a été menée pour illustrer et valider le modèle. Leurs résultats montrent que le modèle stochastique est plus performant que le modèle déterministe, surtout lorsque les ressources de production sont suffisantes. Dans [Quezada 2022], un état de l'art des approches de *lot-sizing* stochastique sans contraintes de capacité avec arbres de scénarios et une amélioration de la méthode *stochastic dual dynamic integer programming algorithm* de [Zou 2019] est proposée pour résoudre ce problème.

Dans [Ben-Tal 2005], les auteurs proposent de résoudre le problème d'optimisation robuste pour un problème de *lot-sizing* avec un seul produit et des incertitudes de forme convexe en se limitant à des recours affines, comme détaillé au Chapitre 1. Dans [Santos 2018], une borne inférieure pour un problème de *lot-sizing* robuste sans contraintes de capacité avec budget d'incertitude sur la demande est obtenue de manière efficace par relaxation des contraintes de non anticipation. Finalement dans [Alem 2018] les auteurs présentent une étude du *General Lot-Sizing and Scheduling Problem* (GLSP) –qui combine un problème de *lot-sizing* et un problème d'ordonnancement– sous demande incertaine à l'aide d'une optimisation robuste à base de budget d'incertitude et d'un modèle de programmation stochastique à deux étapes avec recours. Ils développent une procédure systématique basée sur la simulation de Monte Carlo pour comparer les deux modèles en termes de robustesse et de temps de calcul. Les expérimentations numériques couvrent différentes caractéristiques d'instances, en tenant en compte des budgets d'incertitude et des niveaux de variabilité pour le modèle d'optimisation robuste, ainsi que d'un nombre croissant de scénarios et de fonctions de distribution de probabilité pour le modèle de programmation stochastique. Une autre approche robuste est proposée dans [Roels 2006] où les auteurs étudient une version min-max regret du problème. En particulier, ils s'intéressent au problème avec des informations limitées sur la distribution de la demande. En ce sens, ils quantifient la valeur de l'information supplémentaires et mettent en avant que le fait de savoir que la distribution de la demande a une forme régulière est généralement plus utile que de connaître sa moyenne et sa variance. Cependant, il n'existe, à notre connaissance, pas de travaux –hormi [Guillaume 2020] sur lequel nous nous appuyons dans ce chapitre– traitant du cas particulier où l'incertitude porte sur la demande cumulée.

5.3 Méthode de Benders adverse (BA)

Dans cette section, nous décrivons précisément la méthode de Benders adverse qui permet de résoudre des problèmes d'optimisation robuste avec le critère de robustesse absolue, et l'appliquons au problème de *lot-sizing* avec incertitude sur la demande. Le principe de la méthode de Benders adverse est le suivant. L'idée est de résoudre le problème **minmax** (ou problème maître), d'abord sur un seul scénario (la demande nominale $\hat{\mathcal{D}}$), puis en augmentant le nombre de scénarios considérés. Pour calculer le prochain scénario $\mathcal{D}'$ à ajouter au problème maître, on effectue une phase adverse, c'est-à-dire que l'on cherche, étant donnée la solution particulière S du problème calculée à l'itération précédente, à calculer le pire scénario pour cette solution. Ce problème est appelé problème adverse (que l'on notera **Adv**). Une fois ce scénario calculé, il est ajouté à la liste des scénarios du problème

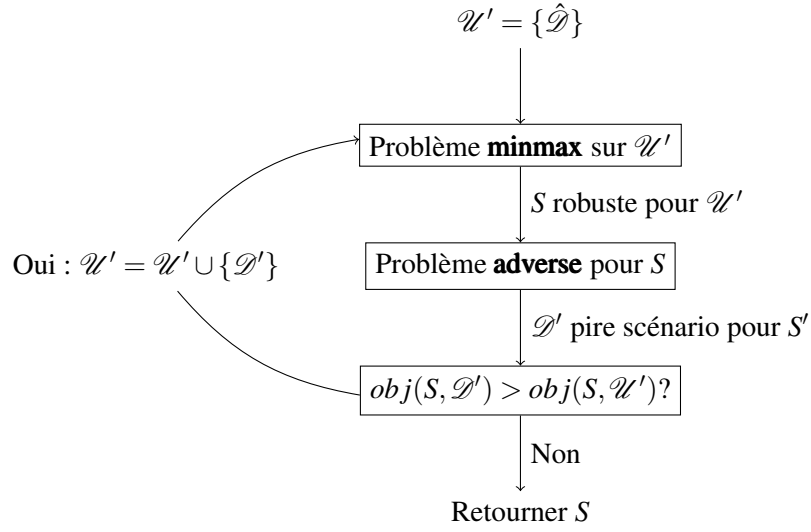


FIGURE 5.1 – Schéma de la méthode de Benders adverse

maître. On réitère alors jusqu'à ce que le critère d'arrêt soit atteint, à savoir quand le scénario obtenu à l'issue de la résolution du problème adverse ne permet pas de dégrader la valeur objectif du problème maître de l'itération en cours. Quand cela se produit c'est que la dernière solution calculée en résolvant le problème **minmax** est la solution robuste recherchée. On peut s'en convaincre en remarquant que par définition, la valeur objectif d'une solution robuste d'un problème **minmax**, ne peut être dégradée par aucun scénario. Le schéma montré sur la figure 5.1 donne une représentation visuelle de cet algorithme.

L'utilisation de cette méthode est recommandée lorsque le problème d'optimisation robuste sur l'ensemble des scénarios est trop difficile pour s'y attaquer frontalement, ce qui peut être dû à la forme de l'ensemble des scénarios considérés, ou à leur nombre en cas d'ensemble de scénarios discrets, et lorsque le problème adverse est relativement facile, ce qui, comme on le verra, est le cas pour notre problème. Pour appliquer cette méthode à notre problème, il "suffit" donc d'être capable de résoudre le problème **minmax** pour une liste de scénarios donnée, et de résoudre le problème **Adv** pour une solution du problème donnée.

5.3.1 Le problème **minmax**

En ce qui concerne le problème **minmax**, on peut le modéliser par le programme linéaire en variables mixtes avec les variables suivantes. Pour une liste de scénario $U \subseteq \mathcal{U}$ et un horizon de temps T on a, pour $1 \leq t \leq T$ et $1 \leq l \leq |U|$, X_t la production cumulée jusqu'au temps t , y_t la variable booléenne qui vaut 1 si une production est effectuée au temps t et 0 sinon, $I_{t,l}$ la quantité de produit stockée au temps t dans le scénario l , $B_{t,l}$ la quantité de produit non vendue au temps t dans le scénario l et $s_{t,l}$ la quantité de produit vendue au temps t dans le scénario l . On peut remarquer que seules les deux premières familles

de variables sont indépendantes de la liste de scénarios. On peut alors écrire le programme linéaire en variables mixtes suivant :

$$\begin{aligned}
& \min z \\
& \text{s.t. } \sum_{t=1}^T (c^I I_{t,l} + c^B B_{t,l} + c^P y_t - b^P s_{t,l}) \leq z & l \leq |U| & (5.1) \\
& B_{t,l} + X_t = D_t^l + I_{t,l} & 1 \leq t \leq T, l \leq |U| & (5.2) \\
& \sum_{i=1}^t s_{i,l} = D_t^l - B_{t,l} & 1 \leq t \leq T, l \leq |U| & (5.3) \\
& X_t - X_{t-1} \leq M y_t & 2 \leq t \leq T & (5.4) \\
& X_1 \leq M y_1 & & (5.5) \\
& B_{t,l}, I_{t,l}, s_{t,l} \geq 0 & 1 \leq t \leq T, l \leq |U| \\
& X \in \mathbb{X} \\
& y_t \in \{0, 1\} & 1 \leq t \leq T
\end{aligned}$$

avec $M = D_T + \Gamma$ et z une variable auxiliaire et D_t^l la demande cumulée au temps t dans le l -ième scénario de la liste U . On notera ce programme linéaire $\mathcal{P}_U$. Les contraintes 5.2 et 5.3 garantissent la cohérence des variables X , B , I et s à chaque pas de temps et pour chaque scénario. On s'assure également de la correction des valeurs des variables y à l'aide des contraintes 5.4 et 5.5. Le "big M " choisi, à savoir $D_T + \Gamma$ est bien une borne supérieure pour les variables X puisqu'il s'agit de la demande maximale possible. Notons que la contrainte $X \in \mathbb{X}$, où $\mathbb{X}$ est un polytope de dimension T permet de représenter des contraintes de capacité ou bien d'autres relations linéaires entre les variables de production de différentes périodes. Finalement, pour un l donné, la quantité à gauche dans l'inégalité 5.1 vaut la valeur de la fonction objectif de la solution X dans le l -ième scénario de la liste U . De cette manière, puisque l'on cherche à minimiser z qui est un majorant de chacune de ces quantités, on résout bien le problème de la forme $\min_{X \in \mathbb{X}} \max_{u \in U}$ souhaitée.

Cette formulation est compacte pour un nombre de scénarios fixé et possède $O(T|U|)$ contraintes, $O(T|U|)$ variables continues et $O(T)$ variables entières. De plus, puisqu'elle s'appuie sur une liste de scénarios, elle a l'avantage de ne pas dépendre de la forme de l'ensemble de scénarios. On peut cependant remarquer qu'elle fait intervenir des contraintes de type "big M " qui ont tendance à poser des difficultés aux solveurs, en fonction du choix de M . Dans notre cas, notre choix dépend de Γ ce qui implique que l'on peut s'attendre à ce que, pour une même instance et une même liste de scénario, une augmentation du budget d'incertitude ralentissent la résolution du problème.

5.3.2 Le problème **Adv**

Quant au problème **Adv**, dans [Guillaume 2020], les auteurs ont montré que ce problème était NP-Difficile au sens faible en le réduisant à une variante du problème sac-à-dos (*knapsack*), et ont proposé un algorithme de programmation dynamique de complexité pseudo-polynomiale pour le résoudre. L'idée de cet algorithme est de calculer un plus long

chemin dans un graphe partitionné construit à partir d'un plan de production X de la manière suivante.

Définition 5 (Graphe de budget). *On note ce graphe $\mathcal{G} = (V, A)$. L'ensemble des noeuds V partitionné en $T + 2$ niveaux et, mis à part le premier et le dernier niveau qui contiennent un unique noeud chacun, chaque niveau contient $\Gamma + 1$ noeuds. On note respectivement v_0 et v_{T+1} les noeuds du premier et du dernier niveau, et $v_{t,i}$ pour $1 \leq t \leq T$ et $0 \leq i \leq \Gamma$ tous les autres noeuds du graphe. L'ensemble des arcs A est construit de sorte que l'arc de $v_{t-1,i}$ vers $v_{t,j}$ existe si et seulement si $i \leq j$ et $j - i \leq \Delta_t$ pour chaque t (ou niveau) compris entre 1 et T , et un arc de $v_{t,i}$ vers $v_{t+1,i}$ est défini pour i compris entre 0 et Γ . Ce graphe est tel que chaque chemin de v_0 vers v_{T+1} représente un scénario de $\mathcal{U}$, et que si un noeud $v_{t,i}$ est sur ce chemin, alors i unités de budget ont été consommées entre la première période et la période t . Ensuite, on définit les coûts des arcs de sorte qu'ils représentent chacun la contribution à la fonction objectif de la décision de faire varier la demande à une période donnée pour une solution X donnée. Plus formellement, on définit les coûts des arcs par :*

$$c_{t-1,i,t,j} = \begin{cases} \max\{c^I(X_t - (\hat{D}_t - (j - i))), c^B(\hat{D}_t + (j - i) - X_t)\} + c^P y_t & \text{si } 1 \leq t \leq T - 1, \\ \max\{c^I(X_t - (\hat{D}_t - (j - i))) - b^P(\hat{D}_t - (j - i)), \\ c^B(\hat{D}_t + (j - i) - X_t) - b^P X_t\} + c^P y_t & \text{si } t = T, \\ 0 & \text{si } t = T + 1 \end{cases}$$

où $c_{t-1,i,t,j}$ est le coût de l'arc entre les noeuds $v_{t-1,i}$ et $v_{t,j}$ lorsque cet arc existe, et $y_t = 0$ si $X_{t-1} = X_t$ et 1 sinon.

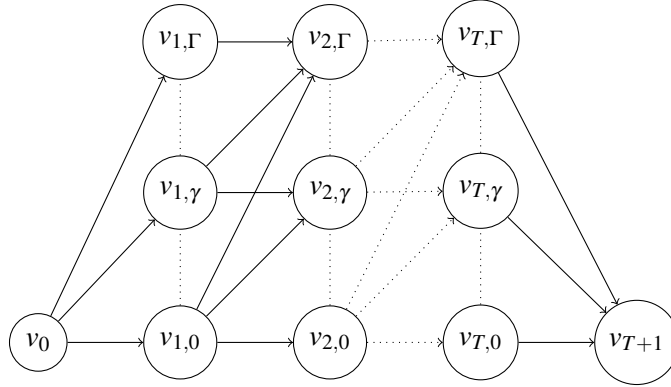


FIGURE 5.2 – Schéma générique du graphe de budget

Sauf précision contraire, dans la suite de ce chapitre, lorsqu'on parlera de "chemin" dans un graphe de budget, on parlera par abus de langage uniquement des chemins entre les noeuds v_0 et v_{T+1} .

On peut montrer que sous l'hypothèse que toutes les données du problème sont entières, l'ensemble des chemins dans ce graphe est en bijection avec l'ensemble des scénarios restreint aux valeurs entières $\mathcal{U} \cap \mathbb{Z}^T$. Pour reconstruire un scénario $D = (D_t)_{t \leq T}$ à partir d'un chemin du graphe de budget, on pose, pour chaque arc $(v_{t-1,i}, v_{t,j})$ sur le chemin

$D_t = \hat{D}_t + s(t)(j - i)$ avec, pour $1 \leq t \leq T - 1$, $s(t) = 1$ si $c^B(\hat{D}_t + (j - i) - X_t) \geq c^I(X_t - (\hat{D}_t - (j - i)))$ et $s(t) = -1$ sinon, et, pour $t = T$ $s(t) = 1$, si $c^B(\hat{D}_t + (j - i) - X_t) - b^P X_t \geq c^I(X_t - (\hat{D}_t - (j - i))) - b^P(\hat{D}_t - (j - i))$ et $s(t) = -1$ sinon. Réciproquement pour un scénario entier $D = (D_t)_{t \leq T}$ donné, on note γ_t la quantité de budget d'incertitude utilisée à la période t dans D , à savoir $|\hat{D}_t - D_t|$. Alors par construction le chemin passant par les arcs $(v_{t-1,i}, v_{t,j})$ pour $1 \leq t \leq T + 1$ et $0 \leq i, j \leq \Gamma$ tels que $|j - i| = \gamma(t)$ existe dans $\mathcal{G}$.

Puisqu'un chemin dans le graphe représente un scénario unique, le graphe $\mathcal{G}$ représente un ensemble de scénarios que l'on note $\mathcal{U}_{\mathcal{G}}$. On a alors $\mathcal{U} \cap \mathbb{Z}^T = \mathcal{U}_{\mathcal{G}}$. Pour une solution X donnée, et donc un coût des arcs de $\mathcal{G}$ donné, on peut montrer (voir [Guillaume 2020]) que par définition du coût des arcs, le coût d'un chemin est égal à la valeur objectif de X dans le scénario représenté par ce chemin. Ainsi, toujours sous l'hypothèse de données entières, il est montré dans [Guillaume 2020]) que le plus long chemin dans le graphe de budget et le coût de ce chemin représentent respectivement le pire scénario dans l'ensemble continu $\mathbb{U}$ pour une solution X et la valeur objectif de cette solution sur ce pire scénario. Ainsi, la résolution du problème **Adv** revient à résoudre un problème de calcul de plus long chemin dans un graphe orienté sans cycle, ce qui peut être résolu dans notre cas en $O(T\Gamma^2)$ par un algorithme de programmation dynamique. On peut ensuite retrouver précisément le scénario par *backtracking*.

5.3.3 Algorithme de Benders adverse pour le lot-sizing incertain

Maintenant que nous avons présenté comment résoudre les problèmes **minmax** et **Adv**, on peut résoudre notre problème en utilisant la méthode de Benders adverse en suivant l'algorithme 9 qui précise le schéma de la figure 5.1 présenté au début de cette section.

Algorithme 9 Benders adverse

```

 $U \leftarrow \{\hat{\mathcal{D}}\}$ 
 $(X, v) \leftarrow$  solution optimale et valeur objectif optimale de  $\mathcal{P}_U$ 
 $(\mathcal{D}, v') \leftarrow$  pire scénario et sa valeur objectif du problème Adv pour la solution  $X$ 
tant que  $v \neq v'$  faire
     $v \leftarrow v'$ 
     $U \leftarrow U \cup \{\mathcal{D}\}$ 
     $(X, v) \leftarrow$  solution optimale et valeur objectif optimale de  $\mathcal{P}_U$ 
     $(\mathcal{D}, v') \leftarrow$  pire scénario et sa valeur objectif du problème Adv pour la solution  $X$ 
fin tant que
retourne  $X$ 

```

5.4 Méthode de Benders adverse étendue (BAE)

Dans cette section, nous proposons maintenant une variante de la méthode de Benders adverse présentée dans la section précédente. L'idée de cette variante est qu'à chaque phase adverse, on ne renvoie pas seulement le pire scénario pour une solution donnée, mais un ensemble de mauvais scénarios. Cependant, le problème maître $\mathcal{P}_U$, défini pour une liste

de scénarios U , passe difficilement à l'échelle lorsque le nombre de scénarios explose. Pour pallier cela, on propose de réécrire le problème maître en se basant cette fois non pas sur une liste de scénarios, mais sur le graphe de budget introduit dans la section précédente.

5.4.1 Graphe partiel de budget

Une manière de voir le graphe de budget est de le considérer comme une représentation relativement compacte de l'ensemble des scénarios. En se basant sur la définition du graphe de budget du chapitre précédent, on définit un graphe partiel de budget, qui représente un sous-ensemble de scénarios, de la manière suivante.

Définition 6 (Graphe partiel de budget). Soit $\mathcal{G} = (V, A)$ un graphe de budget défini suivant la définition 5. On dit que $\mathcal{G}' = (V, A')$ est un graphe partiel de budget de $\mathcal{G}$ si $A' \subset A$ et pour tout arc $a \in A'$, il existe un chemin de v_0 vers v_{T+1} passant par l'arc a . Les poids des arcs de ce graphe sont calculés de la même manière que pour le graphe de budget défini précédemment. On a alors $\mathcal{U}_{\mathcal{G}'} \subset \mathcal{U}_{\mathcal{G}}$.

Nous allons utiliser ce type de graphe pour redéfinir le problème maître.

5.4.2 Problème maître

Tout comme le problème maître de la méthode de Benders adverse classique se basait sur l'approximation de l'ensemble de scénarios $\mathcal{U}$ par une liste de scénarios, nous proposons cette fois un problème maître où l'on approxime $\mathcal{U}$ par un sous-ensemble de scénarios représenté par un graphe partiel de budget. Pour résoudre ce nouveau problème maître, on utilise une formulation en programme linéaire en variables mixtes qui étant donné un sous-graphe de budget $\mathcal{G}'$, calcule le planning robuste X , par rapport à l'ensemble des scénarios représentés par $\mathcal{G}'$. L'idée de cette formulation est de considérer notre problème comme un problème de plus long chemin, en termes de potentiels, dans le graphe $\mathcal{G}' = (V, A')$ avec des arcs dont le poids dépend des variables de décisions, à savoir X_t et y_t qui représentent encore une fois respectivement la production cumulée jusqu'à un instant t et la décision binaire de produire à cet instant ou non. Les variables de potentiels associées à chaque noeud du graphe, notées $\pi_{t,i}$ pour $v_{t,i} \in V$ sont égales au coût du plus long chemin du noeud v_0 jusqu'au noeud $v_{t,i}$. Pour des questions de lisibilité, un arc $(v_{t-1,i}, v_{t,j})$ sera noté $a_{t,i,j}$. On

notera cette formulation $\mathcal{P}'_{\mathcal{G}'}$ et elle s'écrit de la manière suivante :

$$\min \pi_{T+1}$$

$$\text{s.t. } \pi_{t_j} - \pi_{(t-1)_i} \geq c^I(X_t - (\hat{D}_t - (j-i))) + c^P y_t \quad t \in [1, T-1], a_{t,i,j} \in A' \quad (5.6)$$

$$\pi_{t_j} - \pi_{(t-1)_i} \geq c^B(\hat{D}_t + (j-i) - X_t) + c^P y_t \quad t \in [1, T-1], a_{t,i,j} \in A' \quad (5.7)$$

$$\begin{aligned} \pi_{t_j} - \pi_{(t-1)_i} &\geq c^I(X_t - (\hat{D}_t - (j-i))) + c^P y_t \\ &\quad - b^P(\hat{D}_t - (j-i)) \quad t = T, a_{t,i,j} \in A' \end{aligned} \quad (5.8)$$

$$\pi_{t_j} - \pi_{(t-1)_i} \geq c^B(\hat{D}_t + (j-i) - X_t) + c^P y_t - b^P X_t \quad t = T, a_{t,i,j} \in A' \quad (5.9)$$

$$\pi_{T+1} - \pi_{T,i} \geq 0 \quad 0 \leq i \leq \Gamma \quad (5.10)$$

$$\pi_0 = 0, \pi_v \in \mathbb{R} \quad v \in V \quad (5.11)$$

$$X_t - X_{t-1} \leq M y_t \quad 2 \leq t \leq T \quad (5.12)$$

$$X_1 \leq M y_1 \quad (5.13)$$

$$y_t \in \{0, 1\} \quad 1 \leq t \leq T$$

$$X \in \mathbb{X}$$

avec $M = D_T + \Gamma$. Les termes à droite des inégalités des contraintes de 5.6 à 5.11 représentent les coûts des arcs $a_{t,i,j}$ lorsqu'ils existent et bornent les valeurs des variables π . Comme précédemment, les contraintes 5.12 et 5.13 assurent la cohérence entre les variables X et y . Cette formulation comporte $O(|A'| + \Gamma)$ contraintes, $O(|A'| + \Gamma)$ variables continues et $O(T)$ variables entières. Elle a donc l'avantage d'être compacte alors qu'elle permet de résoudre le problème **minmax** pour un nombre possiblement exponentiel de scénarios, ce que la formulation précédente du problème ne permettait pas de faire. On peut remarquer que la taille de l'ensemble des arcs A' est bornée par un $O(T\Gamma^2)$. Cependant, si l'on a choisi d'exprimer le nombre de contraintes et de variables en fonction de $|A'|$ c'est parce que l'on essaiera en pratique de jouer sur cette valeur pour utiliser au mieux notre méthode.

5.4.3 Sous-problème (Adv)

On peut maintenant considérer le problème de la phase adverse, qui cette fois consiste, étant donné un plan de production X , à calculer un graphe partiel de budget $\mathcal{G}'$ qui représente un sous-ensemble de mauvais scénarios pour X . Pour faire cela, on calcule dans un premier temps par programmation dynamique -de la même manière que pour la méthode de Benders adverse classique présentée dans la section précédente- le plus long chemin entre chaque noeud et le noeud v_0 du graphe de budget $\mathcal{G} = (V, A)$. Comme dans la formulation PLNE de la sous-section précédente, on appelle $\pi_{t,i}$ ce coût pour le noeud $v_{t,i} \in V$. On sélectionne ensuite les arcs du graphe partiel de budget par backtracking. On part du noeud v_{T+1} et on considère l'ensemble des noeuds de la T -ième couche qui participent activement au plus long chemin, c'est à dire tels que $\pi_{T+1} = \pi_{T,i} + c_{T,i,T+1}$. On en sélectionne un sous-ensemble, et pour chacun des noeuds sélectionnés on ajoute l'arc dont il est l'origine au graphe partiel. On réitère en remontant le graphe couche par couche jusqu'au noeud v_0 . Ainsi, le nombre d'arcs du graphe partiel dépend de la manière dont on a sélectionné les noeuds à chaque étape du *backtracking*. Les méthodes que nous avons implémentées pour

sélectionner les noeuds déterminant les arcs du graphe partiel de budget sont détaillées dans la section 5.5 de ce chapitre. L'algorithme 10 donne le pseudo code qui permet de résoudre le problème **Adv** à partir d'un graphe de budget dont le coûts des arcs est obtenu à partir d'une solution X donnée.

Algorithme 10 Adv (BAE)

Entrée: Un graphe de budget $\mathcal{G} = (V, A)$

```

 $\pi_0 \leftarrow 0$ 
pour  $t = 1 \rightarrow T + 1$  faire
  pour  $j = 0 \rightarrow \Gamma + 1$  faire
     $\pi_{t,j} \leftarrow \max \{ \pi_{t-1,i} + c_{t-1,i,t,j} \mid (v_{t-1,i}, v_{t,j}) \in A \}$ 
  fin pour
fin pour
 $A' = \{ \}$ 
 $v_{T+1} \leftarrow \text{marqué}$ 
pour  $t = T \rightarrow 1$  faire
   $A'_t = \{ \}$ 
  pour  $j = 0 \rightarrow \Gamma + 1$  faire
    si  $t = T$  et  $\pi_{T+1} = \pi_{T,j} + c_{T,j,T+1}$  alors
       $A'_t \leftarrow A'_t \cup \{ (v_{T,j}, v_{T+1}) \}$ 
    fin si
     $A'_t \leftarrow \text{Selection}(A'_t)$ 
    pour  $(v, v') \in A'_t$  faire
       $v \leftarrow \text{marqué}$ 
    fin pour
    si  $v_{t,j}$  est marqué alors
      pour  $i = 0 \rightarrow \Gamma + 1$  faire
        si  $\pi_{t-1,i} + c_{t-1,i,t,j} = \pi_{t,j}$  alors
           $A'_t \leftarrow A'_t \cup \{ (v_{t-1,i}, v_{t,j}) \}$ 
        fin si
      fin pour
       $A'_t \leftarrow \text{Selection}(A'_t)$ 
      pour  $(v, v') \in A'_t$  faire
         $v \leftarrow \text{marqué}$ 
      fin pour
    fin si
  fin pour
retourne  $\mathcal{G}' = (V, A'), \pi_{T+1}$ 

```

5.4.4 Fusion de graphes partiels de budget

Avant de pouvoir adapter cette étape de phase adverse pour une approche de Benders adverse, il reste une dernière étape. En effet dans l'approche classique, on ajoute les scénarios un à un dans une liste. Dans notre cas, cela signifie que l'on a besoin d'une

opération d'union ou de fusion de graphes. On propose donc la fonction de fusion de graphes suivante : $Merge((V, A'), (V, A'')) = (V, A' \cup A'')$. On peut cependant remarquer que l'ensemble des scénarios représentés par la fusion de deux graphes partiels de budget peut-être plus grand que la somme des tailles des sous-ensembles de scénarios de chacun des deux graphes partiels. Plus formellement, $|\mathcal{U}_{Merge(\mathcal{G}', \mathcal{G}'')}| \geq |\mathcal{U}_{\mathcal{G}'}| + |\mathcal{U}_{\mathcal{G}''}|$. On peut s'en convaincre simplement en regardant la figure 5.3. Sur cette figure est représenté un graphe partiel de budget avec deux chemins. Le premier, $c^1 = v_0 \rightarrow v_{1,1} \rightarrow v_{2,1} \rightarrow v_{3,1} \rightarrow v_4$ représenté avec des tirés, et le second $c^2 = v_0 \rightarrow v_{1,0} \rightarrow v_{2,1} \rightarrow v_{3,2} \rightarrow v_4$ représenté avec des pointillés. Supposons que ce graphe soit la fusion de deux graphes partiels de budget qui contenaient tous les deux un unique chemin, respectivement c^1 et c^2 . On peut voir que le résultat de cette fusion contient deux chemins supplémentaires, à savoir $v_0 \rightarrow v_{1,1} \rightarrow v_{2,1} \rightarrow v_{3,2} \rightarrow v_4$ et $v_0 \rightarrow v_{1,0} \rightarrow v_{2,1} \rightarrow v_{3,1} \rightarrow v_4$. Malgré cela, en utilisant cette opération, on ne change ni la correction ni la terminaison de la méthode de Benders adverse, l'important étant que le graphe contienne au moins le pire scénario pour la solution de l'itération en cours. Cependant, l'existence de ce phénomène pousse à être prudent sur la taille des sous-graphes de budget issus de la phase adverse : si l'on prend des sous-graphes trop grands on risque de se retrouver au bout de quelques itérations à avoir un graphe partiel de budget qui est en réalité le graphe de budget en entier.

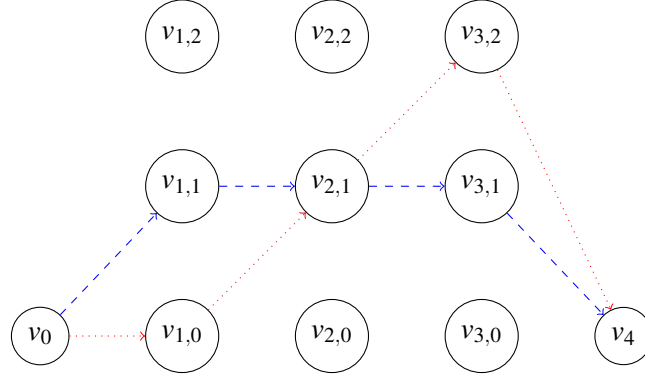


FIGURE 5.3 – Exemple de fusion de graphes partiels de budget

5.4.5 Algorithme de Benders adverse étendue pour le lot-sizing incertain

On peut maintenant présenter l'algorithme de Benders adverse étendu, qui est décrit dans l'algorithme 11. De la même manière que l'on initialisait la liste de scénarios du Benders adverse classique avec le scénario nominal $\hat{D}$, on initialise cet algorithme avec le graphe partiel de budget qui ne représente que le scénario nominal, c'est-à-dire un unique chemin $v_0 \rightarrow v_{1,0} \rightarrow v_{2,0} \rightarrow \dots \rightarrow v_{T,0} \rightarrow v_{T+1}$, que l'on note $\mathcal{G}$.

Algorithme 11 Benders adverse étendu

```

 $\mathcal{G}' \leftarrow \mathcal{G}$ 
 $(X, v) \leftarrow$  solution optimale et valeur objectif optimale de  $\mathcal{P}'_{\mathcal{G}'}$ 
 $(\mathcal{G}'', v') \leftarrow$  graphe partiel de budget et son plus long chemin, solution de Adv pour  $X$ 
tant que  $v \neq v'$  faire
     $v \leftarrow v'$ 
     $\mathcal{G}' \leftarrow \text{Merge}(\mathcal{G}', \mathcal{G}'')$ 
     $(X, v) \leftarrow$  solution optimale et valeur objectif optimale de  $\mathcal{P}'_{\mathcal{G}'}$ 
     $(\mathcal{G}'', v') \leftarrow$  graphe partiel de budget et son plus long chemin, solution de Adv pour  $X$ 
fin tant que
retourne  $X$ 

```

5.5 Expérimentations numériques

5.5.1 Description des instances

Afin de comparer les différentes méthodes, nous avons sélectionné des instances de **csplib.org**. Ces instances comportent entre 200 et 500 périodes de temps et décrivent un problème de lot-sizing multi-produit avec un coût de stockage variant avec le temps. Pour adapter ces instances à notre problème, nous avons tout d'abord agrégé tous les produits et leurs demandes associées en un seul produit. Pour les différents coûts de la fonction objectif, le coût de stockage c^I de la première période de l'instance initiale a été choisi puis étendu à toutes les périodes. Tous les autres coûts (c^B , c^P , et b^P) ont été tirés aléatoirement en suivant une loi uniforme dans l'intervalle $[10, 20]$. En ce qui concerne l'ensemble des scénarios $\mathcal{U}$, celui-ci dépend des valeurs δ_t qui sont, pour chaque pas de temps t l'écart maximal possible à la valeur nominale $\hat{D}_t$. Ils ont chacun été tirés uniformément dans l'intervalle $[0, \hat{D}_t]$. Finalement, On définit le polytope $\mathbb{X}$ avec des contraintes de capacité $X_t \leq \mathbb{X}_t, \forall t \leq T$ en tirant uniformément les valeurs $\mathbb{X}_t$ dans l'intervalle $[\frac{\hat{D}_{t+1} - \hat{D}_t}{2}, \frac{3(\hat{D}_{t+1} - \hat{D}_t)}{2}]$.

5.5.2 Sélection des noeuds

Comme expliqué précédemment la méthode de Benders adverse étendue nécessite, lors de la résolution du problème **Adv** d'une méthode de sélection des arcs qui constitueront le graphe partiel de budget renvoyé par l'algorithme 10. Lors de nos expérimentations, nous avons utilisé deux méthodes de sélection différentes :

- La sélection *SelectAll* consiste à choisir tous les arcs entrants du nœud examiné dont les nœuds d'origine appartiennent à un des plus longs chemins dans le graphe partiel de budget. Autrement dit, en utilisant cette méthode dans l'algorithme 10, les chemins contenus dans le graphe partiel obtenu sont tous les plus longs chemins de ce graphe. Ils constituent l'ensemble de tous les pires scénarios entiers pour la solution courante.
- La sélection *SelectFew* consiste à choisir systématiquement un premier arc et éventuellement un autre avec une probabilité égale à $\frac{1}{T}$ parmi tous les arcs entrant dont les nœuds d'origines appartiennent à un des plus longs chemins dans le graphe

partiel de budget.

Comparativement à la méthode *SelectAll*, le graphe partiel de budget obtenu en utilisant la méthode *SelectFew* représente aussi un sous-ensemble de scénarios pire cas pour la solution courante, mais de taille beaucoup plus réduite. La raison à cela est qu'il existe pour nos instances un grand nombre de pire scénarios équivalents. Expérimentalement, nous avons pu remarquer qu'après plusieurs itérations -et opérations de fusions- le nombre d'arcs du graphe partiel de budget obtenu avec la méthode *SelectAll* avait tendance à exploser rapidement, rendant très vite ce graphe partiel très dense et difficile à exploiter. Ce phénomène explique en partie les résultats expérimentaux de la section qui suit.

5.5.3 Résultats expérimentaux

Les différents programmes linéaires ont été résolus avec CPLEX 12.9 et tous les calculs ont été effectués avec un processeur Intel Xeon CPU E5-2695 v4 2.10GHz sous Linux Ubuntu 16.04.4. Pour ces expérimentations, nous avons considéré deux paramètres. Tout d'abord le budget d'incertitude Γ qui prend ses valeurs dans l'ensemble $\{1, 20, 40, 60, 80, 100\}$, ainsi que notre méthode de sélection. Nous avons au total trois méthodes :

BA : la méthode de Benders adverse standard.

BAE_{all} : la méthode de Benders adverse étendue, en utilisant la méthode de sélection *SelectAll*.

BAE_{few} : la méthode de Benders adverse étendue, en utilisant la méthode de sélection *SelectFew*.

Les deux critères qui sont observés sont le nombre d'itérations de l'algorithme et le temps de calcul pour chacune des méthodes et chacun des Γ . Les résultats sont affichés dans la table 5.1 avec à gauche dans chaque cellule le nombre d'itérations moyen, et à droite le temps de calcul moyen de ces méthodes sur l'ensemble des instances. Pour chaque Γ , les valeurs des cellules avec le plus petit nombre d'itération moyen ou le plus petit temps de calcul moyen ont été mises en gras.

$\Gamma \setminus$ Méthode	<i>BA</i>		<i>BAE_{all}</i>		<i>BAE_{few}</i>	
	iter.	temps	iter.	temps	iter.	temps
1	3.27	1.63	1.82	0.19	1.82	0.19
20	6.18	10.34	1.82	7.82	3.82	1.79
40	4.81	12.62	1.81	29.36	3.90	6.29
60	6.81	21.18	1.90	62.2	5.72	18.30
80	8.09	42.89	1.90	90.4	4.09	21.14
100	5.81	28.45	2.0	134	5.18	44.0

TABLE 5.1 – Nombre d'itérations et temps d'exécution (en secondes) moyen pour chacune des trois méthodes.

Tout d'abord, on peut observer que quand le budget d'incertitude Γ vaut 1 les résultats de *BAE_{all}* et *BAE_{few}* sont similaires. On peut expliquer cela en remarquant qu'avec un budget d'incertitude aussi faible, le nombre de scénarios est également petit –en $O(T)$ – et que,

dans ce cas, les graphes partiels de budget obtenus pendant les phases adverses ne doivent pas beaucoup varier en fonction de la méthode de sélection utilisée. Sans surprise, la méthode BAE_{all} est meilleure que les deux autres quand on regarde le nombre d'itérations en moyenne, et ce pour toutes les valeurs de Γ . Comme expliqué précédemment, avec la méthode de sélection *SelectAll*, on se retrouve en pratique avec un graphe partiel de budget qui recouvre quasiment le graphe de budget total du problème après seulement une ou deux itérations, c'est-à-dire que quasiment l'intégralité des scénarios de $\mathcal{U}_g$ est contenue dans le graphe partiel de budget. C'est pour cela que cette méthode converge en aussi peu d'itérations. En revanche, cela se répercute également sur le temps de calcul. La taille de la formulation du problème adverse de la méthode BAE dépend fortement de la taille du graphe partiel de budget qui, lorsque presque tous les arcs sont présents, contient $O(T\Gamma^2)$ arcs. Il était donc prévisible que le temps de calcul augmente très fortement lorsque Γ augmente également. En ce qui concerne la méthode BAE_{few} on peut observer que son nombre d'itérations est toujours supérieur à celui de BAE_{all} et inférieur à celui de BA et ce quelle que soit la valeur de Γ . Encore une fois cela n'est pas surprenant, car c'est précisément l'idée de BAE que d'ajouter au problème maître un ensemble de scénarios plutôt qu'un seul à chaque itération de l'algorithme. On peut également observer que lorsque Γ vaut 1, 20, 40, 60 et 80 les temps de calcul de BAE_{few} sont significativement plus bas que ceux de BAE_{all} , et toujours en dessous de ceux de BA . C'est n'est en revanche pas le cas quand $\Gamma = 100$, où BA est plus rapide que BAE. Cela peut s'expliquer par le fait qu'à partir d'une certaine valeur de Γ , le nombre de scénarios représenté par le graphe partiel de budget - même ceux produits en utilisant *SelectFew* - reste trop important et ralentisse le problème maître. Cependant les résultats suggèrent qu'en contrôlant correctement la quantité de scénarios ajoutée à chaque itération de BAE, il est possible d'améliorer BA, et parfois de manière importante.

5.6 Conclusion et perspectives

Pour conclure, dans ce chapitre nous avons présenté une variante de la méthode de Benders adverse pour résoudre un problème d'optimisation robuste pour le lot-sizing avec budget d'incertitudes sur la demande cumulée. Les résultats expérimentaux sont encourageants et indiquent que notre variante peut être plus intéressante que l'approche classique si le budget d'incertitude n'est pas trop élevé. Nous pensons qu'il est possible d'améliorer les résultats en contrôlant encore plus finement la taille du graphe partiel de budget calculé pendant la phase adverse de la méthode de Benders adverse étendue. Cela pourrait par exemple être fait en utilisant une méthode de sélection dynamique qui ferait varier les probabilités de sélection des prédécesseurs en fonction du nombre d'arcs qui a déjà été sélectionnés et de l'avancement du *backtracking*. Nous pensons également qu'il est possible d'améliorer les résultats en passant par des approches de compilation de connaissances, afin de trouver un langage de représentation des scénarios un peu plus compact que les graphes partiels de budget, et d'adapter notre problème maître afin de gagner en temps de calcul lors de sa résolution.

